PrestaShop on AWS

Step-by-Step Implementation Documentation

Two-tier deployment: EC2 application server + private RDS MySQL database

Free-tier resources only | Region: eu-north-1 (Stockholm)

Prepared by

John Segun A

Submission

Preliminary Assignment for Cyber Security

Date

20 October 2026

Live site

http://13.60.44.227

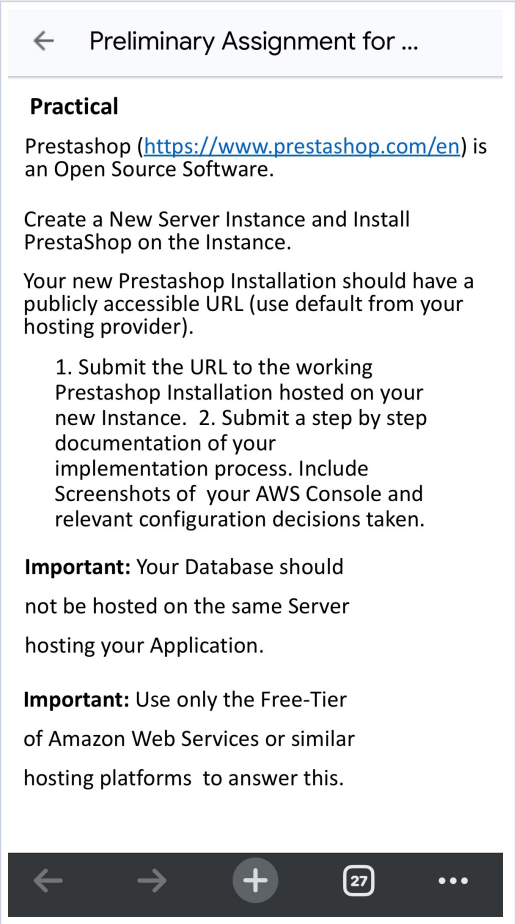
1. Overview

Objective. Deploy PrestaShop, an open-source e-commerce platform, on a new AWS server instance. The database must be hosted on a **separate** server from the application, and only free-tier resources may be used. This document records every step, the configuration decisions taken, and the problems met along the way.

Outcome. A working PrestaShop 9.2.0 store is publicly reachable at **http://13.60.44.227**, the default address provided by AWS.

Component	AWS service	Details
Application server	EC2 instance prestashop-web	Ubuntu Server 24.04 LTS, t3.micro, Apache 2.4, PHP 8.3
Database server	RDS instance prestashop-db	MySQL 8.4, private (no public access), encrypted at rest
Region / network	eu-north-1, default VPC	Both tiers in the same VPC and availability zone

Task brief. The assignment as issued is shown below.


Preliminary Assignment for ...

Practical

Prestashop (https://www.prestashop.com/en) is an Open Source Software.

Create a New Server Instance and Install PrestaShop on the Instance.

Your new Prestashop Installation should have a publicly accessible URL (use default from your hosting provider).

1. Submit the URL to the working Prestashop Installation hosted on your new Instance. 2. Submit a step by step documentation of your implementation process. Include Screenshots of your AWS Console and relevant configuration decisions taken.

Important: Your Database should not be hosted on the same Server hosting your Application.

Important: Use only the Free-Tier of Amazon Web Services or similar hosting platforms to answer this.

Figure 1: Practical assignment brief (Google Docs)

Architecture. Internet users reach the web server on ports 80 and 443. Only the web server can reach the database, on port 3306. The database has no public IP address. Administration is done over SSH (port 22) from a single trusted IP address.

2. Key design decisions

Decision	Reason
RDS MySQL instead of a database on the web server	Meets the requirement that the database must not share a server with the application, and takes database patching off the web server.
Database firewall allows port 3306 only from the web server's security group	Least privilege: only the application tier can talk to the data tier, even if IP addresses change.
Database not publicly accessible	Removes direct internet exposure of the data tier.
SSH restricted to a single IP address (/32)	Stops brute-force attempts against the login port.
Free-tier resources and a zero-spend budget alert	Cost control, as required by the brief.
Encryption at rest on the database	Protects stored data using the AWS-managed KMS key.
PrestaShop 9.2.0 on PHP 8.3	PrestaShop 9 supports PHP 8.3; older 8.x releases target older PHP versions.

3. Step 1: AWS account and cost controls

I created the AWS account (personal details redacted in the screenshot) and worked in the **eu-north-1** region throughout. AWS then set up a first project automatically.

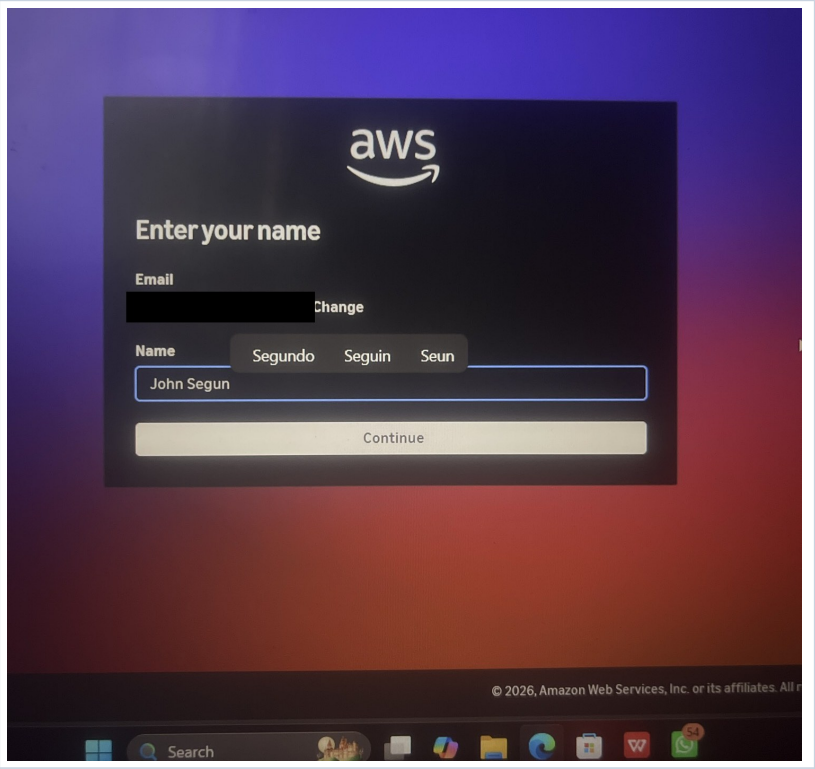


Figure 2: AWS account sign-up (email address redacted)

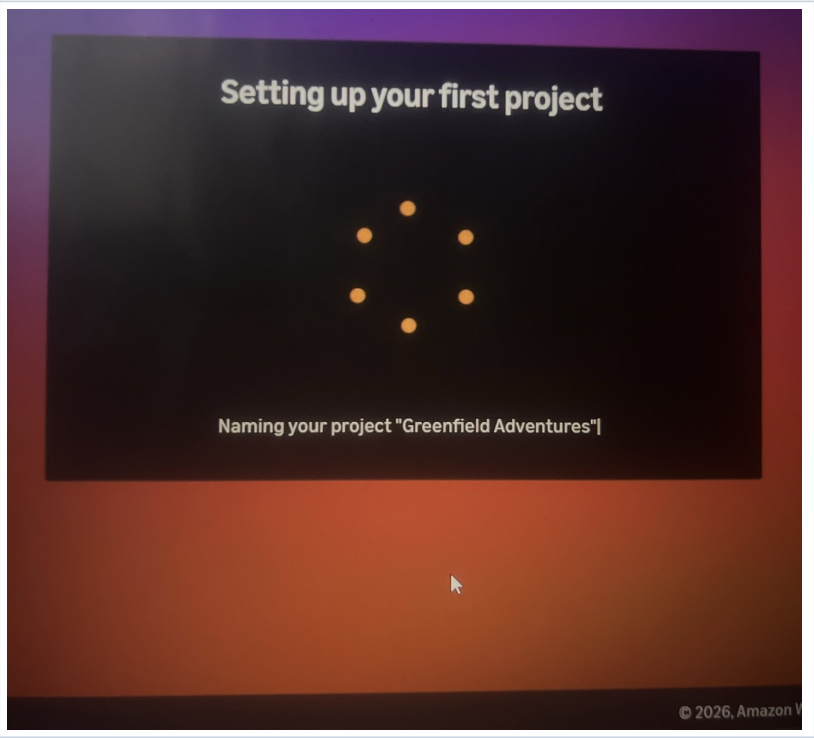


Figure 3: AWS console initial project setup

To avoid unexpected charges, I opened **Billing and Cost Management > Budgets > Create budget** and chose the **Zero spend budget** template. AWS emails me as soon as any spend above $0.01 is incurred.

Screenshot to be inserted
Zero-spend budget created successfully

Figure 4: Zero-spend budget created successfully

4. Step 2: Launch the web server (EC2)

Setting	Value
Name	prestashop-web
AMI	Ubuntu Server 24.04 LTS (Canonical, free tier eligible, 64-bit x86)
Instance type	t3.micro (free tier eligible)
Key pair	prestashop-key2 (RSA, .pem)
Storage	20 GiB gp3
Auto-assign public IP	Enabled
Security group	web-sg
Credit specification	Standard (prevents unlimited CPU bursting charges)

Security group web-sg, inbound rules

Type	Port	Source	Purpose
SSH	22	My IP (single /32 address)	Administration only from my connection

Type	Port	Source	Purpose
HTTP	80	0.0.0.0/0	Public access to the shop
HTTPS	443	0.0.0.0/0	Reserved for future TLS

Screenshot to be inserted
AMI and instance type selection

Figure 5: AMI and instance type selection

Screenshot to be inserted
Security group inbound rules (SSH restricted to My IP)

Figure 6: Security group inbound rules (SSH restricted to My IP)

Screenshot to be inserted
Launch summary and running instance with public IPv4 address

Figure 7: Launch summary and running instance with public IPv4 address

5. Step 3: Create the database server (RDS)

Setting	Value
Engine	MySQL 8.4 (Full configuration)
Template	Free tier
Availability	Single-AZ, no standby
DB instance identifier	prestashop-db
Master username	admin (self-managed strong password, stored privately)
Storage	gp3, 20 GiB, storage autoscaling disabled
Public access	**No**
Initial database name	prestashop
Encryption	Enabled (default KMS key)
Backups	Automated, 1-day retention
Monitoring	Database Insights Standard; Enhanced Monitoring and log exports off

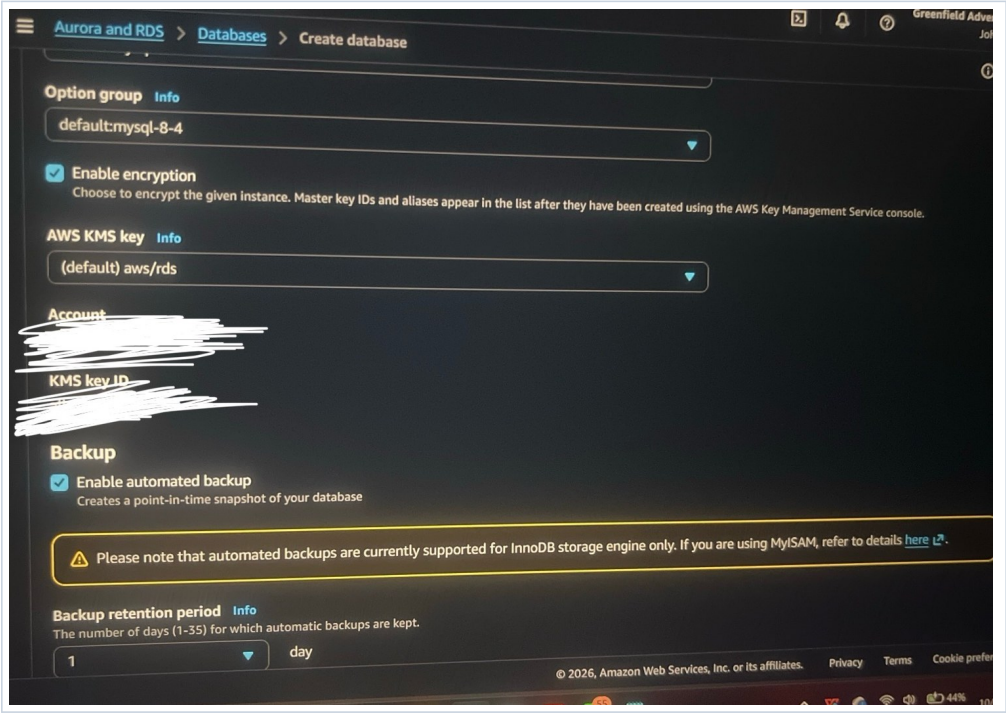


Figure 8: Encryption and backup settings (account number and key ID redacted)

Database firewall (db-sg). The automatic EC2-to-RDS connection step failed with a subnet CIDR conflict, so I created the security group manually under **EC2 > Security Groups**.

Type	Port	Source
MySQL/Aurora	3306	Security group web-sg

I attached **db-sg** to prestashop-db through **RDS > Modify** and removed the default group. The web server is now the only system allowed to reach the database.

Screenshot to be inserted
db-sg inbound rule (MySQL/Aurora 3306 from web-sg)

Figure 9: db-sg inbound rule (MySQL/Aurora 3306 from web-sg)

Screenshot to be inserted
prestashop-db status Available, security group db-sg, publicly accessible No

Figure 10: prestashop-db status Available, security group db-sg, publicly accessible No

6. Step 4: Connect to the server and verify the database

From Windows PowerShell, I restricted the permissions on the private key and connected over SSH.

```
cd $HOME\Downloads
icacls .\prestashop-key2.pem /inheritance:r
icacls .\prestashop-key2.pem /grant:r "$($env:USERNAME):(R)"
ssh -i .\prestashop-key2.pem ubuntu@13.60.44.227
```

- **icacls ... /inheritance:r** removes permissions inherited from the folder; **/grant:r ... (R)** leaves only my user with read access. SSH refuses private keys that other users can read.
- **ssh -i** connects using the key pair; **ubuntu** is the default user on Ubuntu images.

Screenshot to be inserted
Successful SSH login to the Ubuntu server

Figure 11: Successful SSH login to the Ubuntu server

To prove the two-server design works, I connected from the web server to the database:

```
mysql -h <RDS-endpoint> -u admin -p prestashop
```

-h is the database host, **-u** the user, **-p** prompts for the password, and **prestashop** is the database to open. A **mysql>** prompt (server version 8.4.11) confirmed that the db-sg rule lets the web server reach the private database.

Screenshot to be inserted
mysql prompt connected to RDS

Figure 12: mysql prompt connected to RDS

7. Step 5: Install the web stack

```
sudo apt update
sudo apt install -y apache2 php libapache2-mod-php php-mysql php-curl php-gd \
  php-intl php-mbstring php-xml php-zip php-bcmath unzip mysql-client
sudo systemctl enable --now apache2
```

- **apt update** refreshes the package list; **apt install -y** installs without prompting.
- **apache2** is the web server and **libapache2-mod-php** lets it run PHP 8.3.
- The **php-*** packages are the extensions PrestaShop requires: database, HTTP, images, internationalisation, strings, XML, archives and maths.
- **mysql-client** was used to test the database connection.
- **systemctl enable --now** starts Apache immediately and on every reboot.

Apache's default page loaded at the public IP, confirming port 80 is open through web-sg.

Screenshot to be inserted
Apache2 default page at the public IP

Figure 13: Apache2 default page at the public IP

Screenshot to be inserted
apache2 service active (running)

Figure 14: apache2 service active (running)

8. Step 6: Install PrestaShop 9.2.0

The GitHub releases page offers only source code, so I downloaded the official installer package (PrestaShop 9.2.0, basic edition) from the PrestaShop website and uploaded it to the server with **scp**, a secure file copy over SSH.

```
scp -i .\prestashop-key2.pem .\ps.zip ubuntu@13.60.44.227:/tmp/prestashop.zip
```

On the server I unpacked the package and prepared Apache and PHP:

```
cd /tmp
unzip -q prestashop.zip -d ps
sudo rm /var/www/html/index.html
sudo unzip -q /tmp/ps/prestashop.zip -d /var/www/html
sudo chown -R www-data:www-data /var/www/html
sudo a2enmod rewrite
sudo sed -i '/<Directory \/var\/www\/>/,/<\/Directory>/ s/AllowOverride None/AllowOverride All/' /etc/apache
sudo sed -i 's/^memory_limit = .*/memory_limit = 512M/' /etc/php/8.3/apache2/php.ini
sudo systemctl restart apache2
```

- The package holds an inner **prestashop.zip** with the shop files, extracted into Apache's web root **/var/www/html**. The default Apache page is removed so the shop is served at the root URL.
- **chown -R www-data:www-data** gives the Apache user ownership so the installer can write configuration and cache files.
- **a2enmod rewrite** and **AllowOverride All** allow PrestaShop's .htaccess rules and clean URLs.
- The PHP **memory_limit** is raised to 512 MB, as PrestaShop recommends.

I then completed the web installer at http://13.60.44.227: language, license agreement, system compatibility (all checks passed), store information, store content and system configuration. In the database step I entered the RDS endpoint, database name **prestashop**, user **admin** and the master password. The connection test returned **Database is connected**.

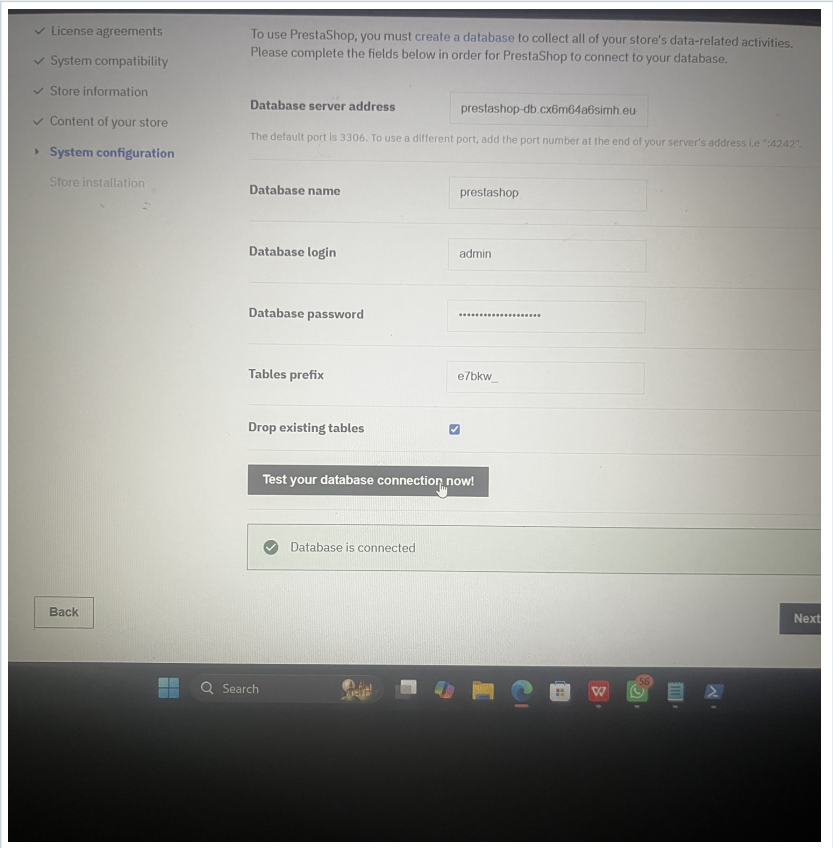


Figure 15: Installer database configuration: Database is connected (password masked)

Screenshot to be inserted
Installer welcome page and system compatibility check

Figure 16: Installer welcome page and system compatibility check

Screenshot to be inserted
Installation finished page (login details hidden)

Figure 17: Installation finished page (login details hidden)

9. Step 7: Post-install hardening

Action	Command or location	Why
Removed the installer folder	sudo rm -rf /var/www/html/install	Stops anyone re-running the installer against the live shop
Back office folder has a random name	Generated by the installer; confirmed with ls /var/www/html | grep admin	Hides the admin login from automated scans
Deleted temporary installer files	rm -rf /tmp/ps /tmp/prestashop.zip	Removes leftovers
Applied pending system updates (190 updates, 153 of them security)	sudo apt update; sudo apt upgrade -y; sudo reboot	Patches known vulnerabilities; a new kernel was installed and loaded by rebooting

Action	Command or location	Why
SSH limited to one IP address	web-sg inbound rule, Source: My IP	Reduces the attack surface
Database private and firewall-restricted	db-sg; public access No	Only the web server can reach port 3306
Rotated an exposed key pair	Created prestashop-key2 and retired the original	The first private key was displayed on screen before launch, so it was replaced and deleted
Encryption at rest	RDS KMS encryption	Protects stored data

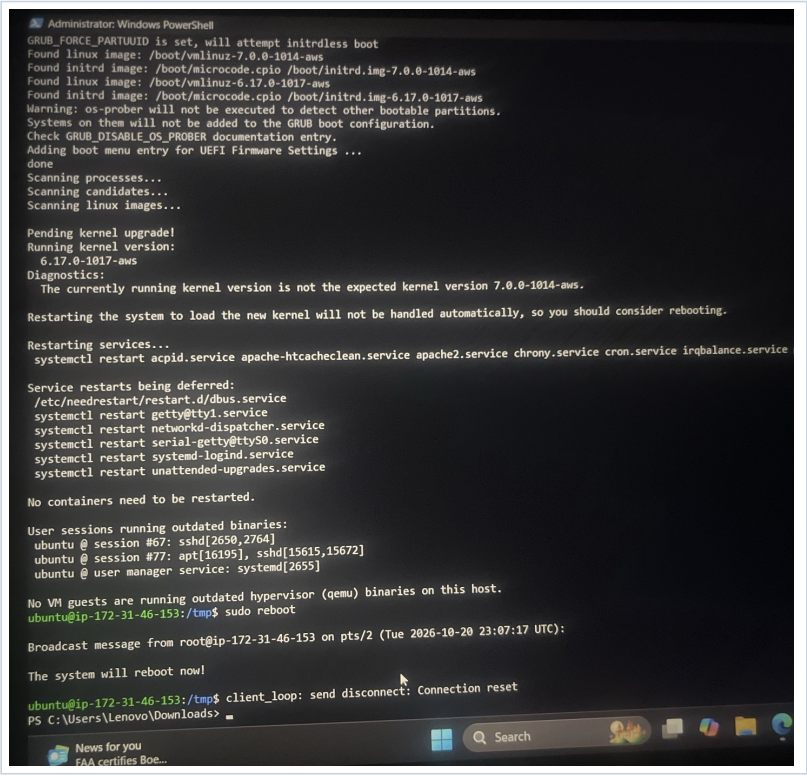


Figure 18: System updates applied and server rebooted to load the new kernel

Screenshot to be inserted
Terminal showing the install folder removed and the admin folder listed

Figure 19: Terminal showing the install folder removed and the admin folder listed

Screenshot to be inserted
Key Pairs page showing only prestashop-key2

Figure 20: Key Pairs page showing only prestashop-key2

10. Challenges encountered and how I resolved them

Problem	Cause	Resolution
Instance launch failed: Microsoft SQL Server is not supported for t3.micro	The AMI I picked was an Ubuntu image bundled with SQL Server	Chose the plain Canonical Ubuntu 24.04 LTS image and reused the existing security group
A default inbound rule opened port 1433 (MSSQL) to the whole internet	Launch wizard default	Replaced it with SSH restricted to My IP
RDS failed to link to EC2 automatically (CIDR conflicts with another subnet)	The auto-created subnet range overlapped an existing one	Created db-sg manually, referencing web-sg as the source
SSH and SCP timed out	My public IP changed, so the SSH rule no longer matched	Updated the SSH rule source to My IP in web-sg
GitHub releases had no installer	Releases attach source code only	Downloaded the official installer package from the PrestaShop website
Wrong engine preselected (Aurora PostgreSQL, Express configuration)	Console defaults	Selected MySQL with Full configuration

11. Verification

- Storefront reachable at **http://13.60.44.227** with demo products.
- Back office dashboard loads and reports PrestaShop 9.2.0.
- Storefront still served correctly after the update reboot, because Apache is enabled at boot.
- Application on EC2 and database on RDS: separate resources with separate security groups.
- Only free-tier resources used; the zero-spend budget is active.

Screenshot to be inserted
Storefront at http://13.60.44.227 (address bar visible)

Figure 21: Storefront at http://13.60.44.227 (address bar visible)

Screenshot to be inserted
Back office dashboard showing PrestaShop 9.2.0 (admin path and token hidden)

Figure 22: Back office dashboard showing PrestaShop 9.2.0 (admin path and token hidden)

12. Production recommendations

This is a short-lived assessment environment. A real deployment would add a registered domain with an HTTPS certificate (the site shows Not secure because it serves plain HTTP on the provider's default URL), a load balancer or web application firewall in front of the web tier, private subnets for the database, credentials in AWS Secrets Manager, multi-AZ and longer backup retention, and monitoring with CloudWatch alarms.

13. Cleanup

After the assessment is reviewed, I will terminate the EC2 instance, delete the RDS instance without a final snapshot, and delete the web-sg and db-sg security groups and the key pair, so that no further charges accrue.